

Recurrent Neural Networks

Naeemullah Khan

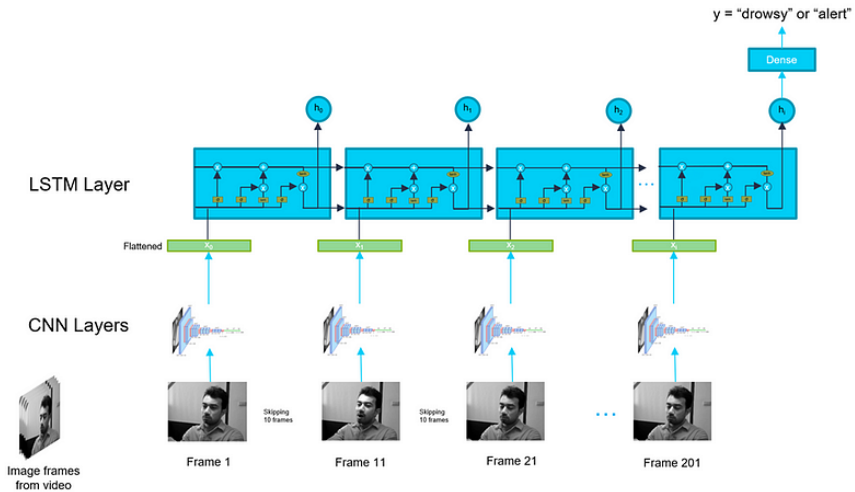
naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 20, 2026



Source: junfengzhang.com

1. Motivation
2. Learning Outcomes
3. Recurrent Neural Networks (RNN)
4. Gated RNNs: LSTM and GRU
5. Image Captioning
6. Sequence-to-Sequence with RNNs
7. Image Captioning with RNNs and Attention
8. Limitations and Future Directions
9. References

Why this topic matters:

- ▶ Many real-world signals are sequential: text, speech, video, and time series.
- ▶ Standard CNNs and MLPs expect fixed-size inputs and ignore order.
- ▶ RNNs process variable-length sequences while sharing weights across time.
- ▶ They are the foundation for sequence-to-sequence learning and the attention mechanism.
- ▶ *"Understanding RNNs and their limits is the gateway to modern Transformers."*

By the end of this session, you will be able to:

- ▶ Describe how an RNN processes sequential data with a recurrent hidden state.
- ▶ Explain the advantages and limitations of vanilla RNNs (e.g., vanishing/exploding gradients).
- ▶ Understand how gated units (LSTM, GRU) mitigate long-term dependency problems.
- ▶ Apply RNNs to vision-language tasks such as image captioning.
- ▶ Describe the sequence-to-sequence framework and the motivation for attention.

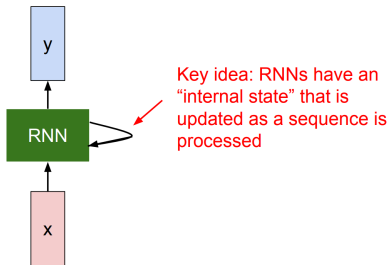
Advanced Computer Vision: **Recurrent Neural Networks (RNN)**

What are RNNs?

- ▶ Neural networks designed for sequence data.
- ▶ Maintain a hidden state h_t that carries information from previous time steps.
- ▶ Recurrent architecture: $h_t = f(h_{t-1}, x_t)$

Use Cases:

- ▶ Language modeling
- ▶ Time series prediction
- ▶ Sequence classification



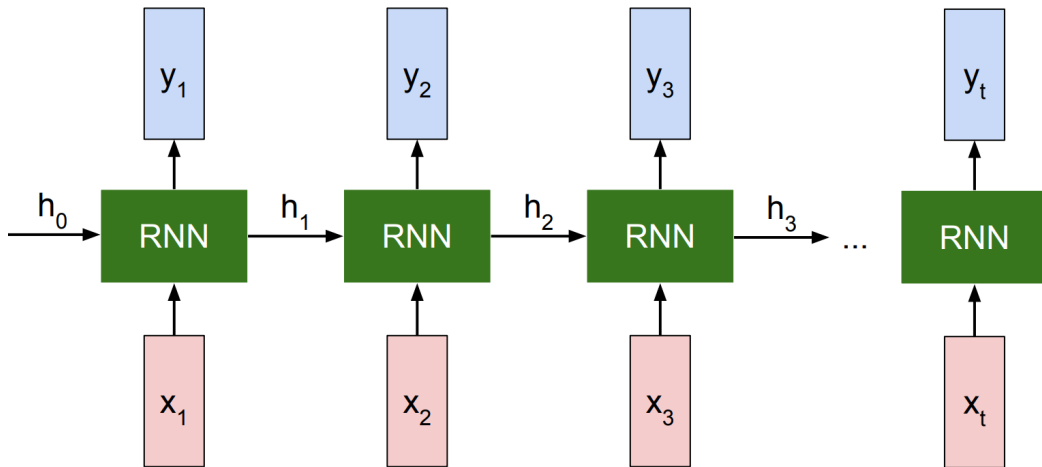
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state / old state input vector at some time step

some function with parameters W

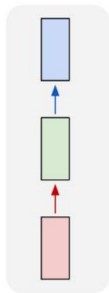
1

Recurrent Neural Networks (RNN) (cont.)

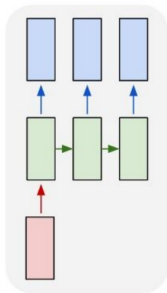


Recurrent Neural Networks (RNN) (cont.)

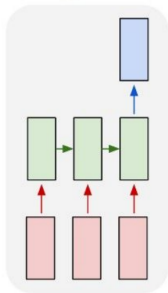
one to one



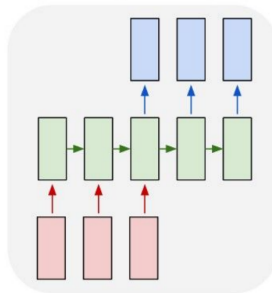
one to many



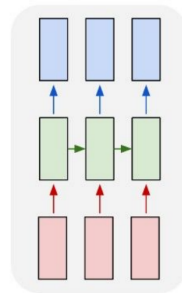
many to one



many to many



many to many



RNN Advantages:

- ▶ Can process any length input
- ▶ Computation for step t can (in theory) use information from many steps back
- ▶ Model size doesn't increase for longer input
- ▶ Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- ▶ Recurrent computation is slow
- ▶ In practice, difficult to access information from many steps back

Limitations of RNNs:

- ▶ Vanishing/exploding gradients in long sequences.
- ▶ Sequential computation — slow to train.
- ▶ Difficulty in capturing long-term dependencies.
- ▶ Cannot parallelize easily due to recursive nature.

Recurrent Neural Networks: **Gated Units: LSTM & GRU**

- ▶ Vanilla RNNs repeatedly multiply by the same weight matrix at every step.
- ▶ Gradients tend to **vanish** (shrink to zero) or **explode** over long sequences.
- ▶ As a result, plain RNNs struggle to connect information across many time steps.
- ▶ **Idea:** add gating mechanisms that let the network *choose* what to keep, forget, and output.

- ▶ Maintains a separate **cell state** c_t that acts as a memory “conveyor belt”.
- ▶ Three gates control the flow of information:
 - **Forget gate** f_t : what to erase from memory.
 - **Input gate** i_t : what new information to store.
 - **Output gate** o_t : what to expose as the hidden state.
- ▶ Additive cell updates create a gradient “highway”, easing long-range learning.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

- ▶ A lighter alternative to the LSTM with **two** gates and no separate cell state.
- ▶ **Reset gate** r_t : how much past state to ignore.
- ▶ **Update gate** z_t : how much to blend old and new state.
- ▶ Fewer parameters $\Rightarrow$ faster to train, often with comparable accuracy.

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Aspect	LSTM	GRU
Gates	3 (input, forget, output)	2 (reset, update)
Separate cell state	Yes	No
Parameters	More	Fewer
Training speed	Slower	Faster
Typical use	Long, complex sequences	Smaller data / faster models

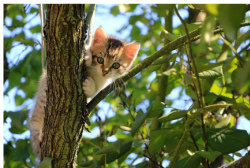
- ▶ Both dramatically outperform vanilla RNNs on long-range dependencies.
- ▶ The choice is empirical — GRU is a strong, cheaper default; LSTM can edge ahead on hard tasks.

Advanced Computer Vision: **Image Captioning**

- A computer Vision task in which we generate captions for images



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



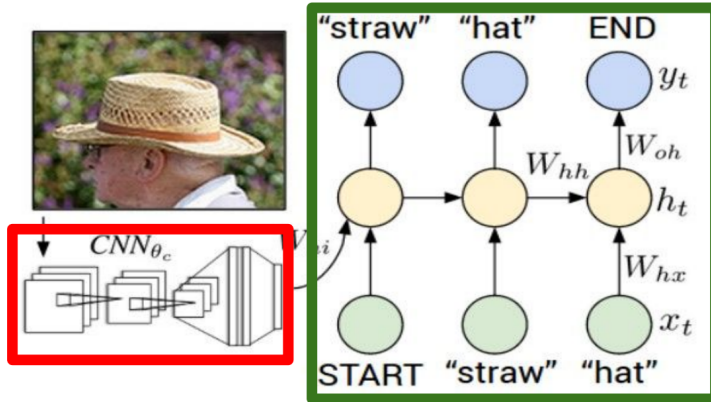
Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

- ▶ The task is to generate a natural language description of the content of an image
- ▶ It combines techniques from computer vision and natural language processing
- ▶ The process typically involves:
 - Extracting features from the image using a convolutional neural network (CNN)
 - Using these features to generate a sequence of words that describe the image, often with a recurrent neural network (RNN) or transformer model

Recurrent Neural Network



Convolutional Neural Network

Advanced Computer Vision: **Sequence-to-Sequence with RNNs**

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

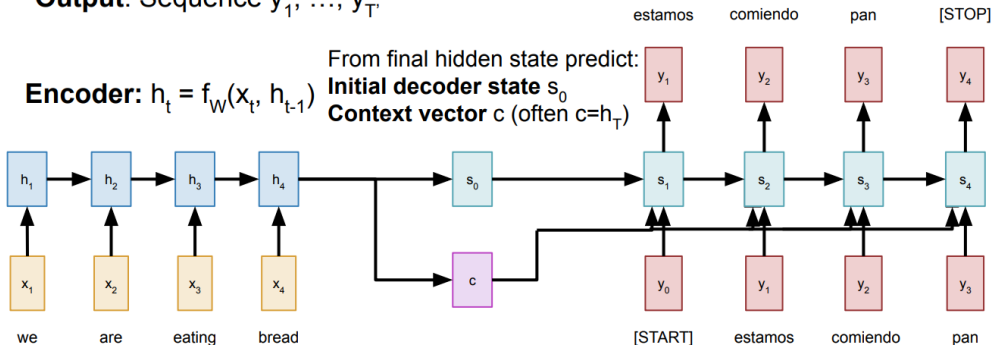
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Sequence-to-Sequence with RNNs (cont.)

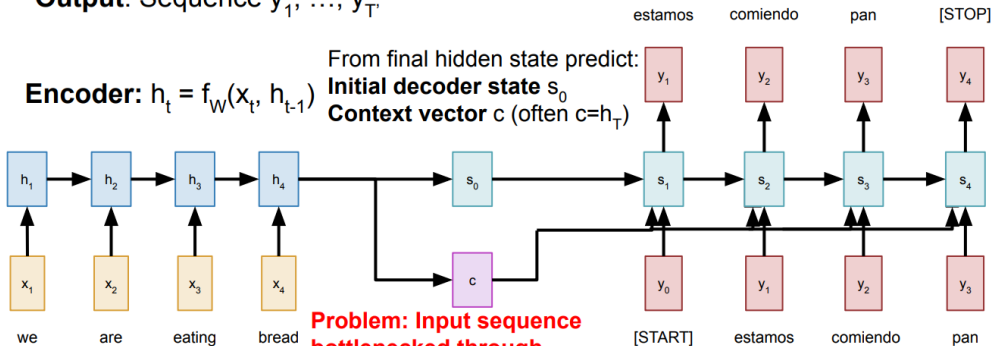
Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c = h_T$)



Problem: Input sequence bottlenecked through fixed-sized vector. What if $T=1000$?

Sutskever et al, "Sequence to sequence learning with neural networks" NeurIPS 2014

Sequence-to-Sequence with RNNs (cont.)

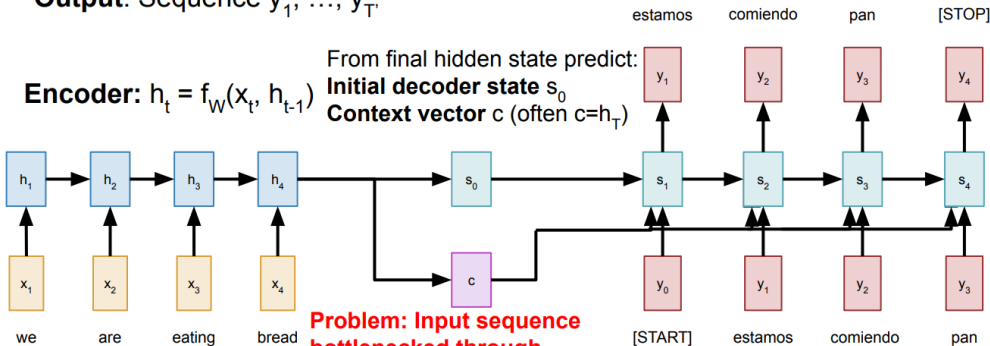
Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)



Idea: use new context vector at each step of decoder!

Sutskever et al, "Sequence to sequence learning with neural networks" NeurIPS 2014

- ▶ **Key idea:** “Don’t process everything — just focus on the most relevant parts.”
- ▶ Attention helps models decide where to look in a sequence.

Equation (simplified):

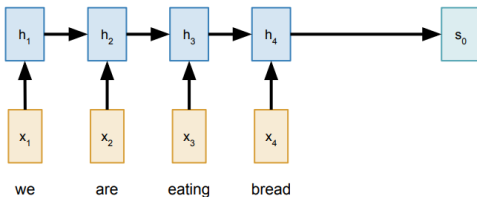
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- ▶ Q = Queries
- ▶ K = Keys
- ▶ V = Values

Input: Sequence $x_1, \dots, x_T$

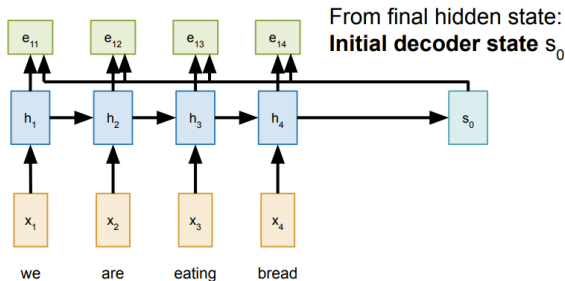
Output: Sequence $y_1, \dots, y_T$

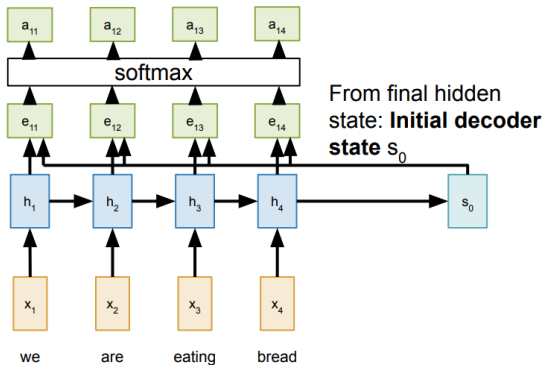
Encoder: $h_t = f_W(x_t, h_{t-1})$ From final hidden state:
Initial decoder state s_0



Compute (scalar) **alignment scores**

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$





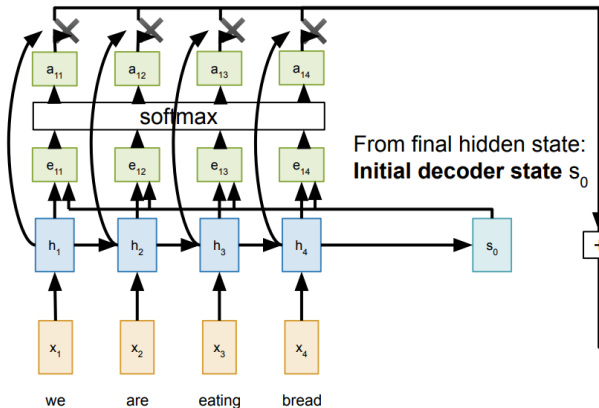
Compute (scalar) **alignment scores**

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$

Normalize alignment scores
to get **attention weights**

$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

Sequence-to-Sequence with RNNs and Attention (cont.)

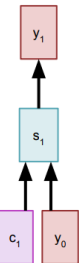


Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores
to get **attention weights**
 $0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

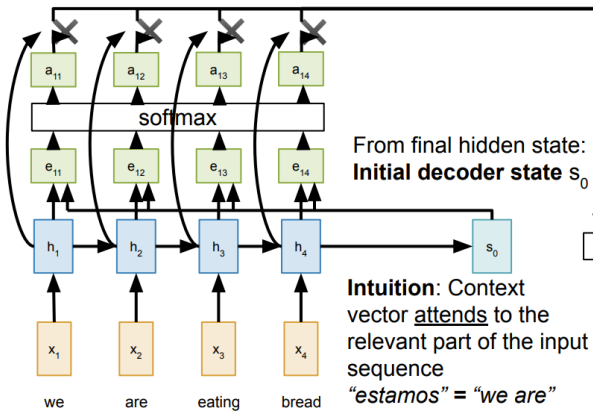
Compute context vector as
linear combination of hidden
states
 $c_t = \sum_i a_{t,i} h_i$

estamos



[START]

Sequence-to-Sequence with RNNs and Attention (cont.)



Compute (scalar) **alignment scores**

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$

Normalize alignment scores to get **attention weights**

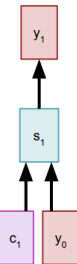
$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

Compute context vector as linear combination of hidden states

$$c_t = \sum_i a_{t,i} h_i$$

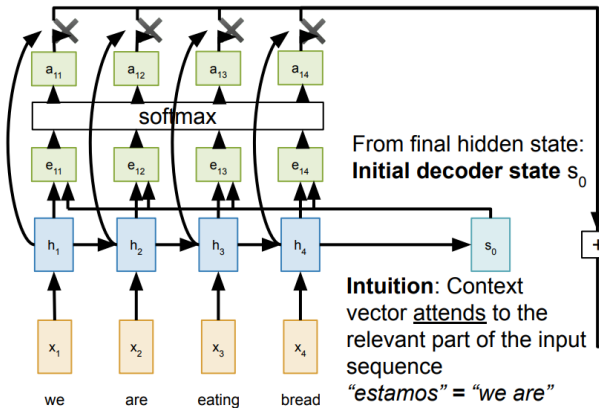
Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

estamos



[START]

Sequence-to-Sequence with RNNs and Attention (cont.)



Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**
 $0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

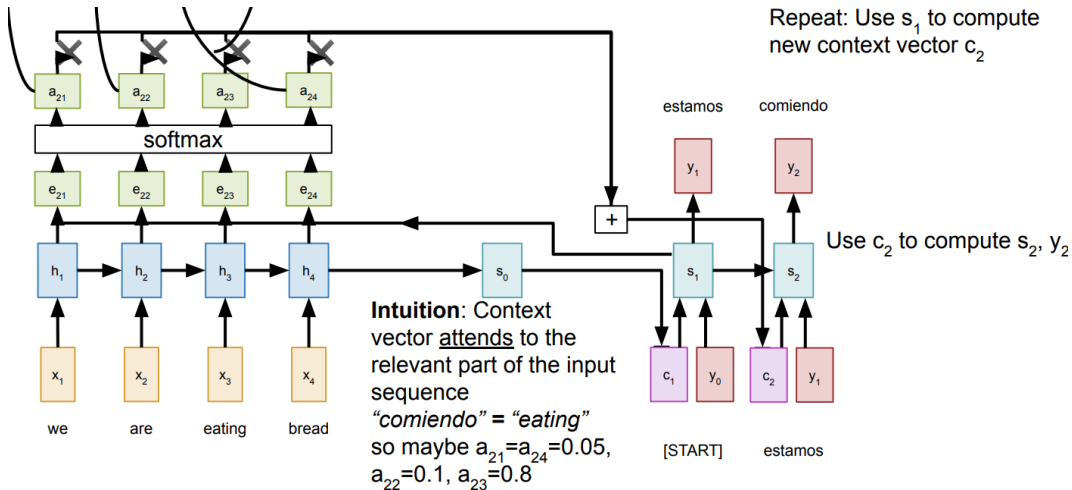
Compute context vector as linear combination of hidden states

$$c_t = \sum_i a_{t,i} h_i$$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

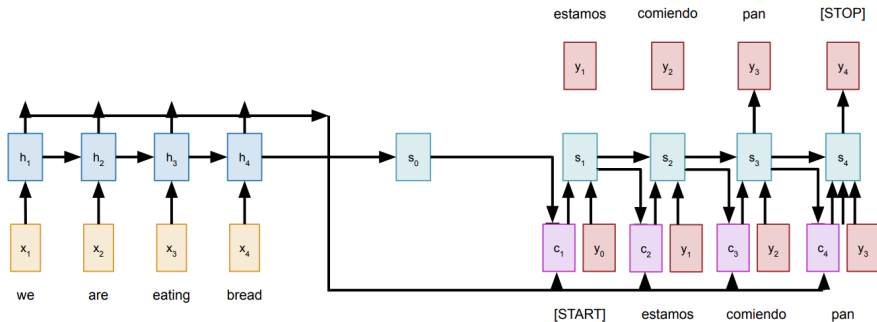
This is all differentiable! No supervision on attention weights – backprop through everything

Sequence-to-Sequence with RNNs and Attention (cont.)



Sequence-to-Sequence with RNNs and Attention (cont.)

- Use a different context vector at each timestep of the decoder.
- The input sequence is not bottlenecked through a single vector.
- At each timestep of the decoder, the context vector "looks at" different parts of the input sequence.



Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

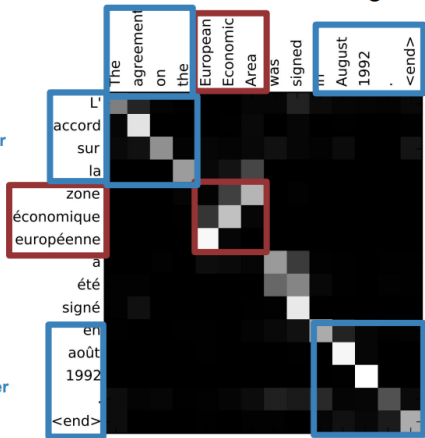
Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Attention figures out different word orders

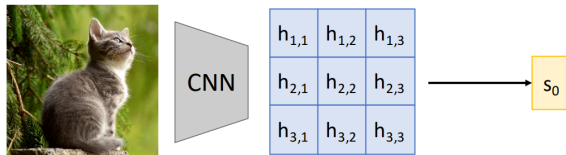
Diagonal attention means words correspond in order

Visualize attention weights $a_{t,i}$



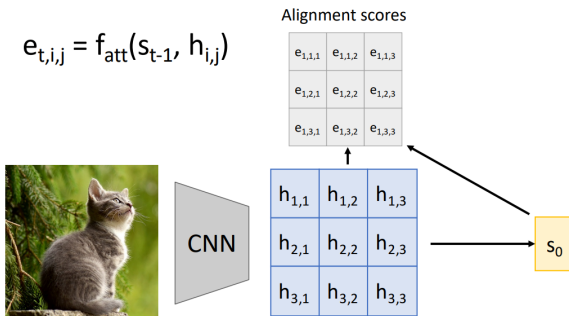
Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

Advanced Computer Vision: **Image Captioning with RNNs and Attention**



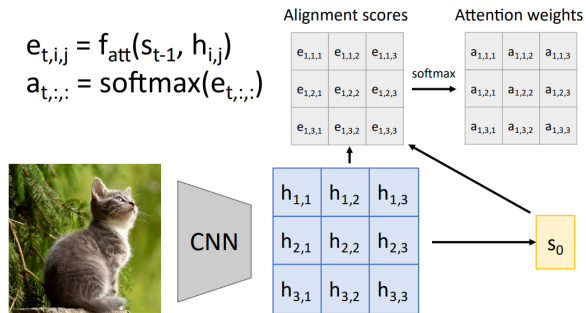
Use a CNN to compute a
grid of features for an image

Image Captioning with RNNs and Attention



Use a CNN to compute a grid of features for an image

Image Captioning with RNNs and Attention



Use a CNN to compute a grid of features for an image

Image Captioning with RNNs and Attention

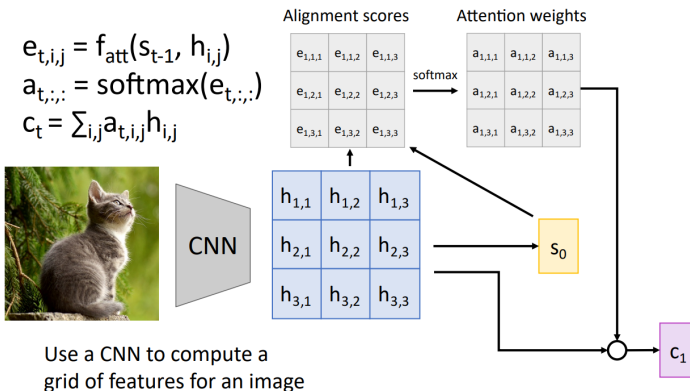
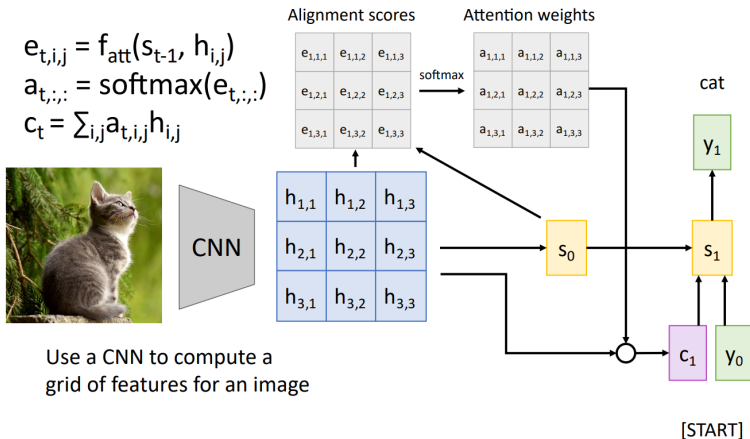
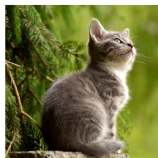


Image Captioning with RNNs and Attention



Use a CNN to compute a grid of features for an image

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$



$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$

Use a CNN to compute a grid of features for an image

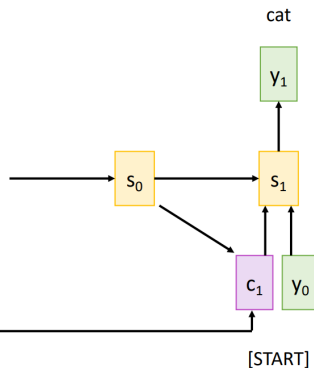


Image Captioning with RNNs and Attention

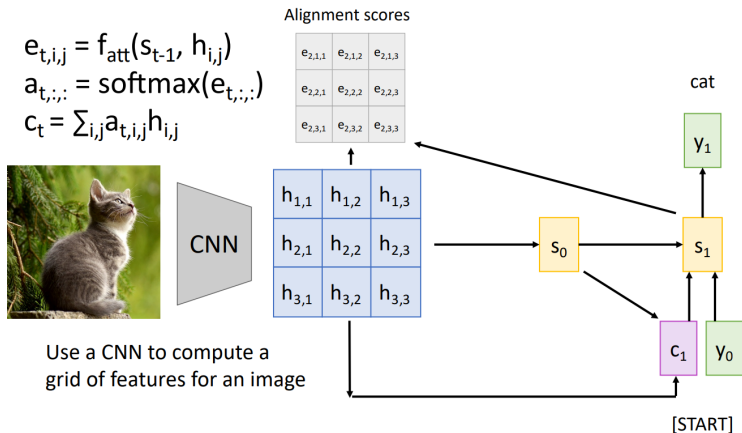


Image Captioning with RNNs and Attention

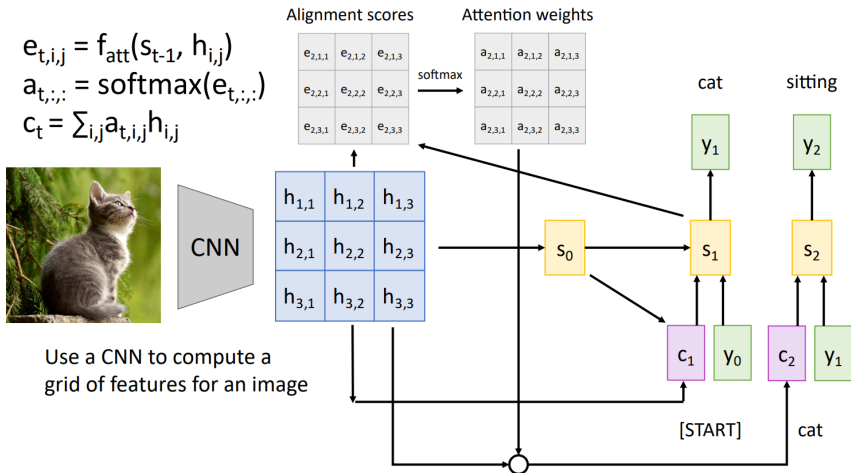
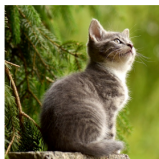


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Each timestep of decoder uses a different context vector that looks at different parts of the input image



$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$

Use a CNN to compute a grid of features for an image

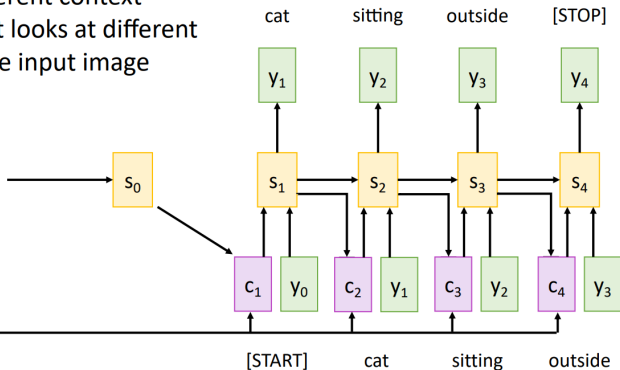


Image Captioning with RNNs and Attention



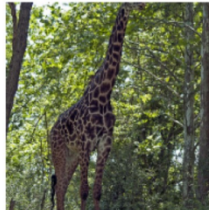
A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.



Recurrent Neural Networks: **Limitations and Future Directions**

- ▶ **Sequential computation:** cannot be parallelized across time steps (slow to train).
- ▶ **Long-range dependencies:** even LSTMs/GRUs struggle on very long sequences.
- ▶ **Limited memory:** the entire context is squeezed into a fixed-size hidden state.
- ▶ **Bottleneck in seq2seq:** a single context vector limits the decoder.

- ▶ **Attention:** let the decoder look at all encoder states ([Bahdanau et al.](#); next topic).
- ▶ **Transformers:** drop recurrence entirely for full parallelism ([Vaswani et al.](#)).
- ▶ **Efficient sequence models:** e.g., state-space models ([S4](#), [Mamba](#)).

Recurrent Neural Networks: **References**

- [1] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [2] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. *EMNLP*. <https://arxiv.org/abs/1406.1078>
- [3] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *Advances in Neural Information Processing Systems*, 27. <https://arxiv.org/abs/1409.3215>
- [4] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *International Conference on Learning Representations*. <https://arxiv.org/abs/1409.0473>

- [5] Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhutdinov, R., Zemel, R., & Bengio, Y. (2015). Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. *International Conference on Machine Learning*.
<https://arxiv.org/abs/1502.03044>
- [6] Fei-Fei Li, Johnson, J., & Yeung, S. Stanford CS231n: Deep Learning for Computer Vision. <http://cs231n.stanford.edu/>